



4. MÄRZ 2024

DYNAMISCHES PREISOPTIMIERUNGSMODELL IM E-COMMERCE

Entwicklung und Implementierung eines dynamischen
Preisoptimierungs-Modells für das E-Commerce-Sortiment am
Beispiel eines international tätigen Branchenführers

ANDREAS TRAUT

Senior Data Scientist geprüft durch das Fraunhofer Institut



Exposé

In einem hochdynamischen E-Commerce-Umfeld, insbesondere auf Plattformen wie Amazon, ist eine agile und datengesteuerte Preisstrategie entscheidend für die Maximierung von Umsatz und Marge sowie die Sicherung der Wettbewerbsfähigkeit. Statische Preismodelle oder manuelle Anpassungen können den schnellen Marktveränderungen, Wettbewerbsaktionen und Nachfrageschwankungen oft nicht gerecht werden. Um das volle Potenzial des E-Commerce-Geschäfts eines international tätigen, mittelständischen Branchenführers auszuschöpfen, wird in dieser Arbeit die Konzeption, Entwicklung und Implementierung eines Machine-Learning-basierten Systems zur dynamischen Preisoptimierung untersucht. Ziel ist es, Preisempfehlungen automatisiert zu generieren, die Preiselastizitäten auf Produktebene berücksichtigen und zur Steigerung von Kennzahlen wie Umsatz (Turnover) und Nettomarge (Net Margin) beitragen.

Die zentrale Herausforderung liegt in der Integration und Verarbeitung heterogener Datenquellen sowie der Komplexität der zugrundeliegenden Modelle. Benötigt werden Verkaufsdaten, Lagerbestandsinformationen, Wettbewerbspreise (z.B. über APIs oder Scraping), Marketingaktivitätsdaten (z.B. aus Pacvue) und potenziell externe Marktdaten. Diese Daten müssen mittels robuster ETL-Prozesse, unter Nutzung von PowerQuery und SQL Server Integration Services (SSIS), in das bestehende Datawarehouse überführt und im Schichtenmodell (Core, Bizcore, Datamart) aufbereitet werden. Eine hohe Datenqualität, beispielsweise durch Abgleich von Pacvue- und internen Daten, ist hierbei essenziell.

Im Kern des Projekts steht die Entwicklung von Machine-Learning-Modellen zur Berechnung von Preiselastizitäten und zur Vorhersage optimaler Preispunkte. Hierfür kommen Python mit Bibliotheken wie scikit-learn, pandas und numpy zum Einsatz. Verschiedene Algorithmen (z.B. Regressionsmodelle, Zeitreihenanalysen) werden evaluiert und mittels historischer Daten trainiert. Die Modellergebnisse müssen validiert werden, beispielsweise durch A/B-Testing-Methodiken im E-Commerce-Kontext.

Ein weiterer Schwerpunkt ist die Operationalisierung der Ergebnisse und die Befähigung des Amazon-Teams sowie des Controllings. Die Preisempfehlungen müssen entweder direkt in Systeme integriert oder über aussagekräftige Dashboards bereitgestellt werden. Hierfür werden auf Basis geeigneter Tabular Modelle im SQL Server Analysis Service (SSAS) interaktive Power BI Reportings erstellt. Diese Visualisierungen, unter intensiver Nutzung von DAX-Berechnungen, ermöglichen nicht nur die Überwachung der Preisstrategie und deren Auswirkungen auf Turnover und Net Margin, sondern auch die Analyse der Modellgüte und der Sensitivität auf verschiedene Einflussfaktoren (z.B. Price Sensitivity Analysen). Die SQL-Expertise (T-SQL, Stored Procedures, Views) ist dabei für die Datenaufbereitung und -bereitstellung im Backend unerlässlich.

Diese Arbeit geht über die reine Datenanalyse („Data Analyst“ Rolle) hinaus, indem sie prädiktive und präskriptive Analytik (siehe Kapitel 1.5) nutzt, um aktiv in die Geschäftssteuerung einzugreifen. Sie demonstriert den Aufbau einer komplexen Data-Science-Lösung von der Datenintegration über die Modellentwicklung bis zur operativen Implementierung und dem Monitoring mittels moderner BI-Tools. Das Ergebnis ist ein intelligentes Preisoptimierungssystem, das die Entscheidungsfindung im E-Commerce auf eine neue Stufe hebt und einen messbaren Beitrag zur Erreichung der strategischen Ziele eines international tätigen, mittelständischen Branchenführers leistet.

Gliederung

1. Einleitung (1 Seite)
 - 1.1 Problemstellung: Limitierungen statischer Preisstrategien im dynamischen E-Commerce Umfeld (insb. Amazon)
 - 1.2 Zielsetzung: Maximierung von Umsatz (Turnover) und Marge (Net Margin) durch ein ML-basiertes Preisoptimierungssystem
 - 1.3 Relevanz und Kontext für einen international tätigen, mittelständischen Branchenführer
 - 1.4 Theoretische Einordnung: Prädiktive und Präskriptive Analytik im Projektkontext
 - 1.5 Cross Industry Standard Process for Data Mining (CRISP-DM)
2. Analyse der Ausgangslage und Datenanforderungen (ca. 1,5 Seiten) [Business Understanding / Data Understanding]
 - 2.1 Bestehende Preisprozesse und Herausforderungen im E-Commerce Team (insb. Amazon)
 - 2.2 Identifikation und Bewertung benötigter Datenquellen
 - 2.2.1 Interne Daten (Verkäufe, Lagerbestand, Produktdaten, ERP-Daten)
 - 2.2.2 Externe Daten (Wettbewerbspreise, Marketing-Plattformdaten z.B. Pacvue, Marktdaten)
 - 2.2.3 Anforderungen an Datenqualität, Granularität und Verfügbarkeit
3. Datenintegration und -aufbereitung für die Preismodellierung (ca. 2,5 Seiten) [Data Understanding / Data Preparation]
 - 3.1 Konzeption der ETL-Strecken zur Datenakquise und -integration
 - 3.1.1 Nutzung von PowerQuery und SQL Server Integration Services (SSIS)
 - 3.1.2 Anbindung von APIs oder Scraping-Lösungen (für Wettbewerbsdaten)
 - 3.2 Aufbau und Erweiterung des Data Warehouse nach Schichtenmodell (Core, Bizcore, Datamart) für Pricing-Daten
 - 3.2.1 Datenmodellierung im Backend (SQL)
 - 3.2.2 Maßnahmen zur Sicherstellung der Datenqualität (z.B. Abgleich Pacvue vs. interne Daten)
 - 3.3 Feature Engineering mittels Python (pandas, numpy) und SQL
 - 3.3.1 Erstellung relevanter Merkmale (z.B. Zeitmerkmale, Preisindizes, Marketing-Einflüsse, rollierende Kennzahlen)
 - 3.3.2 Transformation von Daten für Modellierungszwecke (z.B. Log-Transformation)
4. Entwicklung der Machine-Learning-Modelle zur Preisoptimierung (ca. 3-4 Seiten) [Modeling]
 - 4.1 Auswahl geeigneter Modellierungsansätze
 - 4.1.1 Modelle zur Schätzung der Preiselastizität (z.B. Regressionsmodelle)
 - 4.1.2 Modelle zur Nachfrageprognose (z.B. Zeitreihenanalyse, weitere Regressionsmodelle)
 - 4.2 Implementierung der Modelle in Python
 - 4.2.1 Nutzung von Bibliotheken (scikit-learn, pandas, numpy etc.)
 - 4.2.2 Umgang mit produkt- oder kategoriespezifischen Unterschieden
 - 4.3 Training, Validierung und Auswahl der Modelle
 - 4.3.1 Aufteilung in Trainings-, Validierungs- und Testdaten
 - 4.3.2 Metriken zur Bewertung der Modellgüte
 - 4.4 Ableitung von Preisempfehlungen aus den Modellergebnissen

5. Evaluation und Validierung der Preisstrategie (ca. 2 Seiten) [Evaluation]
 - 5.1 Technische Evaluation der Modellergebnisse und Elastizitäten
 - 5.2 Konzeption und Simulation von A/B-Tests im E-Commerce Kontext
 - 5.3 Bewertung der Auswirkungen auf die Geschäftsziele (Umsatz, Marge)
 - 5.4 Iterative Verbesserung der Modelle basierend auf Evaluationsergebnissen
6. Operationalisierung und Monitoring im BI-System (ca. 3 Seiten) [Deployment]
 - 6.1 Bereitstellung der Modellergebnisse (z.B. Speicherung in DWH, Bereitstellung über API)
 - 6.2 Aufbau eines analytischen Datenmodells (Tabular Model in SSAS) für das Frontend
 - 6.2.1 Definition von Dimensionen, Fakten und Hierarchien für Pricing-Analysen
 - 6.2.2 Implementierung relevanter Kennzahlen mittels DAX (Turnover, Net Margin, Price Sensitivity Indikatoren)
 - 6.3 Entwicklung von Power BI Dashboards und Reports
 - 6.3.1 Visualisierung von Preisempfehlungen und deren Einflussfaktoren
 - 6.3.2 Monitoring der Preisentwicklung und Modell-Performance
 - 6.3.3 Bereitstellung von Analysewerkzeugen für das Amazon-Team und Controlling
 - 6.4 Überlegungen zur Automatisierung des Gesamtprozesses (Datenaktualisierung, Modell-Retraining, Reporting)
7. Zusammenfassung und Ergebnisse (ca. 1 Seite) [Evaluation / Deployment Context]
 - 7.1 Darstellung des entwickelten Systems zur dynamischen Preisoptimierung
 - 7.2 Zusammenfassung der Kernergebnisse (z.B. ermittelte Elastizitäten, Performance des Systems)
 - 7.3 Beitrag zur datengesteuerten Entscheidungsfindung und Erreichung strategischer Ziele
 - 7.4 Ausblick auf mögliche Weiterentwicklungen und Limitationen

1. Einleitung

1.1 Problemstellung: Limitierungen statischer Preisstrategien im dynamischen E-Commerce Umfeld (insb. Amazon)

Der E-Commerce, insbesondere auf dominanten Plattformen wie Amazon, ist durch eine außerordentliche Dynamik gekennzeichnet. Preise von Wettbewerbern ändern sich häufig, Kundenreaktionen sind unmittelbar sichtbar, Marketingkampagnen beeinflussen die Nachfrage und das Produktangebot wird ständig angepasst. Diese hohe Volatilität und Komplexität erfordern von Unternehmen eine hohe Agilität in ihrer Preisstrategie, um wettbewerbsfähig zu bleiben und Ertragspotenziale optimal auszuschöpfen.

Traditionelle Preisstrategien, die auf statischen Preislisten, periodischen manuellen Anpassungen oder einfachen, regelbasierten Ansätzen beruhen, erweisen sich in diesem Umfeld zunehmend als unzureichend. Ihre wesentlichen Limitierungen sind:

- **Reaktionsgeschwindigkeit:** Manuelle Prozesse oder langwierige Abstimmungsrunden sind zu träge, um zeitnah auf kurzfristige Marktveränderungen oder Aktionen von Wettbewerbern reagieren zu können. Chancen zur Umsatzsteigerung oder Margenoptimierung bleiben ungenutzt, oder es wird zu spät auf aggressive Preisunterbietungen reagiert.
- **Komplexität und Skalierbarkeit:** Die Anzahl der Produkte (SKUs), der relevanten Wettbewerber und der Einflussfaktoren auf die Nachfrage (Saisonalität, Marketing, Lagerbestand, Kundenbewertungen etc.) ist oft sehr groß. Eine manuelle Berücksichtigung all dieser Faktoren für jedes Produkt ist kaum leistbar und extrem fehleranfällig.
- **Datennutzung:** Die Fülle an verfügbaren Daten aus Verkaufstransaktionen, Webseiten-Tracking, Marketingplattformen (wie Pacvue) und Wettbewerbsbeobachtung wird bei statischen Ansätzen nicht oder nur unzureichend genutzt. Wertvolle Informationen über Preissensitivitäten und Nachfragemuster bleiben verborgen.
- **Suboptimalität:** Ohne eine systematische Analyse der Preiselastizität und der Nachfrageeffekte führen manuelle oder rein kostenbasierte Preisentscheidungen häufig zu suboptimalen Ergebnissen – entweder werden mögliche Margen verschenkt oder Absatzpotenziale durch zu hohe Preise nicht realisiert.

Diese Unzulänglichkeiten führen dazu, dass Unternehmen mit statischen Preisstrategien im dynamischen E-Commerce-Umfeld systematisch hinter ihren Möglichkeiten zurückbleiben und einem unnötigen Wettbewerbsdruck ausgesetzt sind.

1.2 Zielsetzung: Maximierung von Umsatz (Turnover) und Marge (Net Margin) durch ein ML-basiertes Preisoptimierungssystem

Angesichts der in Kapitel 1.1 beschriebenen Limitierungen statischer Ansätze ist das primäre betriebswirtschaftliche Ziel, das mit der Implementierung eines dynamischen Preisoptimierungssystems verfolgt wird, die Maximierung von Umsatz (Turnover) und Nettomarge (Net Margin) im E-Commerce-Geschäft. Es geht darum, durch intelligentere Preisgestaltung einen messbaren Beitrag zur Steigerung der Profitabilität und des Geschäftsvolumens zu leisten.

Um dieses übergeordnete Geschäftsziel zu erreichen, verfolgt diese Arbeit das spezifische technische und konzeptionelle Ziel, ein Machine-Learning-(ML)-basiertes System zur dynamischen Preisoptimierung zu konzipieren und prototypisch zu implementieren. Dieses System soll

datengestützt und automatisiert Preisempfehlungen für Produkte im E-Commerce-Sortiment generieren.

Der Kern dieses Systems liegt in der Fähigkeit:

1. **Preiselastizitäten zu ermitteln:** Mittels ML-Modellen sollen die Preissensitivitäten für einzelne Produkte oder Produktgruppen auf Basis historischer Verkaufs-, Preis-, Marketing- und Wettbewerbsdaten berechnet werden.
2. **Nachfrage zu prognostizieren:** Die Modelle sollen lernen, wie sich Preisänderungen unter Berücksichtigung anderer relevanter Faktoren (z.B. Saisonalität, Marketingaktivitäten) auf die verkaufte Menge auswirken.
3. **Automatisierte Preisempfehlungen abzuleiten:** Basierend auf den ermittelten Elastizitäten, Nachfrageprognosen sowie definierbaren Geschäftsregeln (z.B. Mindestmargen, Preisobergrenzen, Wettbewerbsorientierung) soll das System konkrete, optimierte Preisvorschläge generieren.

Die prototypische Umsetzung in dieser Arbeit dient dazu, die technische Machbarkeit aufzuzeigen, die Methodik zu validieren und den potenziellen Nutzen eines solchen Systems greifbar zu machen. Sie legt den Grundstein für eine spätere, vollumfängliche Implementierung im produktiven Umfeld des Unternehmens.

1.3 Relevanz und Kontext für einen international tätigen, mittelständischen Weltmarktführer

Die in den vorangegangenen Abschnitten skizzierte Problemstellung statischer Preisstrategien und die Zielsetzung der Entwicklung eines dynamischen, Machine-Learning-basierten Preisoptimierungssystems besitzen eine hohe strategische Relevanz für einen international tätigen, mittelständischen Weltmarktführer. Als etabliertes Unternehmen in verschiedenen Produkt-Bereichen agiert dieses Unternehmen in einem Marktumfeld, das zunehmend durch den Online-Handel und insbesondere durch große Plattformen wie Amazon geprägt wird. Um in diesem hochdynamischen Umfeld nicht nur zu bestehen, sondern auch Wachstumspotenziale voll auszuschöpfen und die Wettbewerbsfähigkeit langfristig zu sichern, sind fortschrittliche, datengesteuerte Ansätze unerlässlich.

Die Implementierung eines intelligenten Preisoptimierungssystems, wie es in dieser Arbeit konzipiert und prototypisch umgesetzt wird, bietet dem international tätigen, mittelständischen Branchenführer konkrete Vorteile:

Maximierung von Umsatz und Marge: Das Kernziel des Systems ist die direkte positive Beeinflussung zentraler betriebswirtschaftlicher Kennzahlen wie Umsatz (Turnover) und Nettomarge (Net Margin). Durch die Berücksichtigung von Preiselastizitäten und Nachfragemustern auf Produktebene können Preise so angepasst werden, dass sie je nach Marktsituation und Produktlebenszyklus entweder den Absatz maximieren oder die Profitabilität steigern.

Steigerung der Wettbewerbsfähigkeit: Im E-Commerce, speziell auf Plattformen mit hoher Preistransparenz wie Amazon, ist die Fähigkeit zur schnellen und fundierten Reaktion auf Wettbewerbsaktionen und Marktveränderungen entscheidend. Ein automatisiertes System ermöglicht es dem international tätigen, mittelständischen Branchenführer, agiler zu agieren als bei manuellen oder statischen Preisprozessen und somit Marktanteile zu sichern oder auszubauen.

Effizienzsteigerung und Ressourcenschonung: Die Automatisierung der Preisfindung reduziert den manuellen Aufwand im E-Commerce-Team (insbesondere dem Amazon-Team) und setzt Ressourcen frei, die für strategischere Aufgaben genutzt werden können.

Förderung einer datengesteuerten Entscheidungskultur: Die Entwicklung und Nutzung eines solchen Systems fördert das Verständnis und die Anwendung von Datenanalysen innerhalb des Unternehmens. Es liefert nicht nur Preisempfehlungen, sondern auch wertvolle Einblicke in das Kundenverhalten (z.B. Preissensitivität), die für Marketing, Produktmanagement und Controlling von Nutzen sind. Das Projekt hebt die Entscheidungsfindung im E-Commerce auf eine neue Stufe.

1.4 Begründung der Abgrenzung zum Data Analysten

Die Konzeption und prototypische Umsetzung dieses Preisoptimierungssystems dient auch als konkrete Begründung für die Notwendigkeit der Schaffung einer dedizierten Data Science Umgebung. Die Komplexität des Vorhabens – von der Integration heterogener Datenquellen über die anspruchsvolle Modellierung von Preiselastizitäten mittels Machine-Learning bis hin zur Operationalisierung und Visualisierung in einem BI-System – übersteigt klar den Aufgabenbereich traditioneller Data-Analystenrollen.

Das Projekt erfordert folgende Kompetenzen:

- **Machine-Learning:** Praktische Erfahrung in der Erstellung von ML-Modellen (z.B. mit Python und Bibliotheken wie scikit-learn) zur Lösung spezifischer Geschäftsprobleme wie Preisoptimierung und Elastizitätsberechnung.
- **Datenmanagement & -modellierung:** Tiefgehende Kenntnisse in ETL-Prozessen (PowerQuery, SSIS), SQL, DWH-Schichtenmodellen und analytischer Datenmodellierung (SSAS Tabular) zur Bereitstellung einer robusten Datengrundlage.
- **Business Intelligence:** Umfassende Erfahrung mit Datenvisualisierungstools wie Power BI und DAX zur Erstellung aussagekräftiger Dashboards und Reports für Fachanwender wie das Amazon-Team und das Controlling.
- **E-Commerce-Verständnis:** Ausgeprägtes Verständnis für die spezifischen Prozesse und KPIs im E-Commerce.

Das Projekt kann wegen folgenden Gründen abgegrenzt werden zu einem Data Analyst:

- **Komplexität:** Dieses Projekt erfordert nicht nur die Erstellung von Berichten, sondern die Entwicklung von Vorhersagemodellen (ML) und Optimierungsalgorithmen.
- **Technologien:** Es nutzt explizit fortgeschrittene Techniken und Werkzeuge (Python/scikit-learn für ML, SSAS für komplexe Modelle, SSIS für ETL), die über typische Analysten-Werkzeuge hinausgehen.
- **Fokus:** Der Schwerpunkt liegt auf prädiktiver (Was wird passieren?) und präskriptiver (Was sollte getan werden?) Analytik, statt primär auf deskriptiver (Was ist passiert?) Analyse.
- **Automatisierung & Systemdesign:** Es geht um den Aufbau eines automatisierten Systems, nicht nur um ad-hoc Analysen oder manuelle Berichte. Dies beinhaltet Aspekte des Data Engineerings und der Softwareentwicklung (Modell-Deployment).
- **Statistische Tiefe:** Erfordert ein tiefes Verständnis statistischer Methoden und Machine-Learning-Konzepte (Preiselastizität, Modellvalidierung, A/B-Testing).

Dieses Projekt fungiert somit als "Leuchtturmprojekt", das exemplarisch den Bedarf und den unmittelbaren Mehrwert eines dedizierten Data-Science-Setups für die Erreichung strategischer Ziele des international tätigen, mittelständischen Branchenführers im wichtigen Wachstumsfeld E-

Commerce aufzeigt. Es demonstriert, wie durch die Anwendung fortgeschrittener Analytik (prädiktiv und präskriptiv) operative Prozesse optimiert und strategische Entscheidungen fundiert werden können. Die Anwendungsmöglichkeiten einer solchen technischen Umgebung geht über das Projektthema „dynamisches Preisoptimierungsmodell im E-Commerce“ hinaus.

1.5 Theoretische Einordnung: Prädiktive und Präskriptive Analytik im Projektkontext

Die Nutzung von **prädiktiver Analytik** in diesem Projekt bedeutet konkret, dass mithilfe von Machine-Learning-Modellen zukünftige Ereignisse vorhergesagt werden – beispielsweise wie sich die Verkaufsmenge bei einer Preisänderung entwickeln wird (Schätzung der Preiselastizität) oder wie hoch die Nachfrage in der nächsten Periode sein könnte. Sie beantwortet die Frage: 'Was wird wahrscheinlich geschehen?'.

Die **präskriptive Analytik** geht einen entscheidenden Schritt weiter und bildet den Kern dieses Projekts zur Preisoptimierung. Sie nutzt die Ergebnisse der prädiktiven Modelle (z.B. die Elastizität), um unter Berücksichtigung definierter Geschäftsziele (wie Umsatz- oder Margenmaximierung) und Rahmenbedingungen (wie Wettbewerbspreise) konkrete Handlungsempfehlungen abzuleiten – hier also die optimalen Preisvorschläge. Sie beantwortet die Frage: 'Was sollten wir tun, um unser Ziel zu erreichen?'. Durch diese Verknüpfung greift das entwickelte System aktiv in die Geschäftssteuerung ein.

Diese fortgeschrittenen Analysestufen bauen auf den grundlegenderen Stufen auf, die ebenfalls für das Geschäftsverständnis relevant sind:

Die **deskriptive Analytik** dient dazu, vergangene Ereignisse zusammenzufassen und zu visualisieren, um einen Überblick über die Geschäftsentwicklung zu geben. Ein typisches Beispiel hierfür ist ein Self-Service BI-System in Power BI, das dem Vertrieb (Sales), Produktmanagement und Controlling Kennzahlen wie den erzielten Umsatz, Deckungsbeitrag oder die verkaufte Menge für verschiedene Produkte und Kunden transparent macht. Sie beantwortet die Frage: 'Was ist passiert?'.

Die **diagnostische Analytik** baut auf der deskriptiven Analyse auf und versucht, die Ursachen für die beobachteten Ergebnisse zu ergründen. Nutzer des genannten Power BI-Systems könnten beispielsweise durch Drill-Downs, Filterung oder Vergleichsanalysen untersuchen, warum der Deckungsbeitrag einer Produktgruppe gesunken ist oder welche Faktoren zu einem Umsatzanstieg bei bestimmten Kunden geführt haben. Sie beantwortet die Frage: 'Warum ist es passiert?'.

Während deskriptive und diagnostische Analysen essenziell für das grundlegende Geschäftsverständnis und die Kontextualisierung sind (und oft durch PowerBI-Systeme wie das erwähnte abgedeckt werden), liegt der Fokus dieser Arbeit klar auf der Entwicklung prädiktiver Modelle und insbesondere präskriptiver Empfehlungen, um proaktiv und datengesteuert die Preisgestaltung zu optimieren.

1.6 Cross Industry Standard Process for Data Mining (CRISP-DM)

Der Cross Industry Standard Process for Data Mining (CRISP-DM) ist ein Standard für Datenanalyse Prozesse - und Herwechseln zwischen zwei verschiedenen Phasen geben.

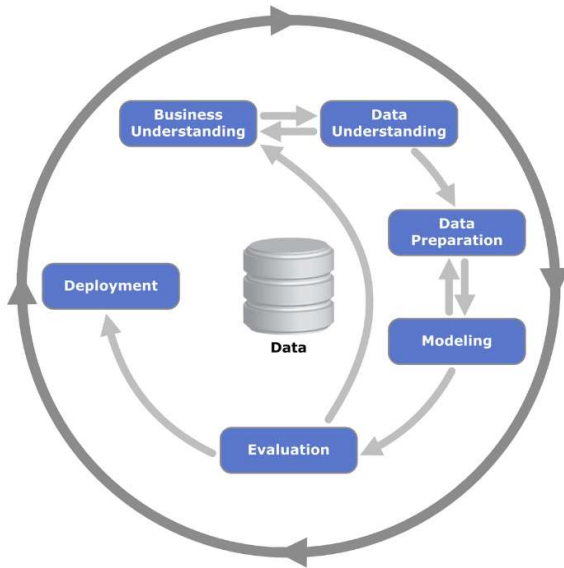


Figure 1-1 Cross Industry Standard Process for Data Mining (CRISP-DM), Quelle: Wikipedia, CC BY-SA 3.0

Die sechs Phasen können, wie folgt, zusammengefasst werden:

- **Business Understanding:** In dieser Phase werden die Anforderungen und Zieldefinition ermittelt sowie eine Beschreibung der unternehmerischen Problemstellung erstellt.
- **Data Understanding:** Diese Phase umfasst die Datenerfassung, die Datenbeschreibung und das Untersuchen und Bewerten der Daten.
- **Data Preparation:** In dieser Phase werden die Daten bereinigt und transformiert in ein Zielformat.
- **Modeling:** Die Modeling Phase beschäftigt sich mit den Modellierungsverfahren. In der vorliegenden Arbeit wird darunter das Verfahren zum Erstellen eines „Datenmodells“ verstanden.
- **Evaluation:** Während dieser Phase werden die Umsetzungen und Prozesse bewertet.
- **Deployment:** Diese Phase beschreibt die Bereitstellung der Implementierung aus der Entwicklungsumgebung in die produktiv genutzte Umgebung.

Nach jedem Kapitel wird die Einordnung in diesen CRISP-DM Standard erklärt.

2. Analyse der Ausgangslage und Datenanforderungen

2.1 Bestehende Preisprozesse und Herausforderungen im E-Commerce Team (insb. Amazon)

Entwurf:

- *Hypothetische Beschreibung der aktuellen (vermutlich manuellen oder regelbasierten) Preisgestaltungsprozesse bei einem international tätigen, mittelständischen Branchenführer.*
- *Identifikation der damit verbundenen Herausforderungen (z.B. Zeitaufwand, Inkonsistenzen, verpasste Chancen).*

2.2 Identifikation und Bewertung benötigter Datenquellen

2.2.1 Interne Daten (Verkäufe, Lagerbestand, Produktdaten, ERP-Daten)

Entwurf:

- *Auflistung der benötigten internen Daten (Verkäufe, Lagerbestand, Produktdaten, ERP-Daten).*

- *Bezug zur Prototyp-Umsetzung: Beschreibung der simulierten internen Daten (CSV-Format, Inhalt, Struktur).*

2.2.2 Externe Daten (Wettbewerbspreise, Marketing-Plattformdaten z.B. Pacvue, Marktdaten)

Entwurf:

- *Auflistung der benötigten externen Daten (Wettbewerbspreise, Marketing-Plattformdaten wie Pacvue, Marktdaten).*
- *Diskussion möglicher Beschaffungswege (APIs, Scraping).*
- *Bezug zur Prototyp-Umsetzung: Beschreibung der simulierten externen Daten (CSV-Format, Inhalt, Struktur, z.B. statische oder leicht variierende Wettbewerbspreise).*

2.3 Anforderungen an Datenqualität, Granularität und Verfügbarkeit

Entwurf:

- *Diskussion der Wichtigkeit von hoher Datenqualität (z.B. Abgleich Pacvue vs. interne).*
- *Festlegung der benötigten Granularität (z.B. tägliche Verkäufe/Preise).*
- *Anforderungen an die zeitliche Verfügbarkeit der Daten für ein operatives System.*
- *Bezug zur Prototyp-Umsetzung: Erläuterung, dass für den Prototyp Qualität und Verfügbarkeit durch die synthetischen Daten als gegeben angenommen wurden.*

3. Datenintegration und -aufbereitung für die Preismodellierung (ca. 2,5 Seiten) [Data Understanding / Data Preparation]

3.1 Konzeption der ETL-Strecken zur Datenakquise und -integration

3.1.1 Nutzung von PowerQuery und SQL Server Integration Services (SSIS)

Entwurf:

- *Beschreibung der geplanten Nutzung von PowerQuery und SQL Server Integration Services (SSIS) im Zielsystem*

3.1.2 Anbindung von APIs oder Scraping-Lösungen (für Wettbewerbsdaten)

Entwurf:

- *Erläuterung der Anbindung von APIs oder Scraping-Lösungen für Wettbewerbsdaten*
- *Prototyp-Umsetzung: Detaillierte Beschreibung, wie die ETL-Strecke im Prototyp vereinfacht umgesetzt wurde:*
- *Laden der synthetischen CSV-Daten mittels Python (pandas).*
- *Alternativ/Ergänzend: Nutzung von Power Query in Power BI Desktop für Transformationen.*
- *Laden der vorbereiteten Daten in die lokale SQL Server Developer Edition Datenbank.*
- *Klarstellung, dass SSIS im Prototyp nicht verwendet wurde.*

3.2 Aufbau und Erweiterung des Data Warehouse nach Schichtenmodell (Core, Bizcore, Datamart) für Pricing-Daten

3.2.1 Datenmodellierung im Backend (SQL)

Ein zentraler Baustein für fortgeschrittene Analysen wie die dynamische Preisoptimierung ist ein robustes und gut strukturiertes Data Warehouse (DWH). Dieses dient als zentrale Sammelstelle, um Daten aus verschiedenen Quellsystemen zu integrieren, langfristig zu speichern und für Analysezwecke aufzubereiten. Im Gegensatz zu transaktionalen Quellsystemen (OLTP), wie ERP-Systeme, die für schnelle Lese- und Schreibvorgänge einzelner Datensätze optimiert sind, sind DWH-Systeme für komplexe Abfragen und Analysen über große Datenmengen ausgelegt.

Für das Data Warehouse, das auf einer Microsoft Azure Cloud SQL-Datenbank basiert, wird eine etablierte, mehrschichtige Architektur verwendet. Diese Struktur fördert die Nachvollziehbarkeit, Wartbarkeit und Flexibilität des Systems. Der Datenfluss folgt dabei einem ELT-Prozess (Extract-Load-Transform), bei dem Daten zunächst extrahiert, dann in das DWH geladen und erst dort transformiert werden. Die wesentlichen Schichten sind:

1. **Staging-Schicht:** Die erste Anlaufstelle für Daten aus den Quellsystemen (z.B. ERP). Hier werden die Rohdaten unverändert abgelegt.
2. **Cleanse-Schicht:** In dieser Schicht erfolgen erste Datenbereinigungen und Validierungen. Fehlerhafte oder unvollständige Daten werden identifiziert und behandelt. Techniken wie Lookups zur Auflösung von Fremdschlüssel-Beziehungen und das Logging von Fehlern kommen hier zum Einsatz, oft gesteuert durch SSIS-Pipelines.
3. **Core-Schicht:** Diese Schicht bildet das Fundament des DWH. Hier werden die bereinigten Daten aus verschiedenen Quellen integriert und in einem stabilen, konsistenten Modell abgelegt. Eine wesentliche Aufgabe ist die Historisierung von Datenänderungen, typischerweise nach dem Prinzip der Slowly Changing Dimensions Typ 2 (SCD II). Dabei wird für jeden Datensatz ein Gültigkeitszeitraum und Status verwaltet, um Änderungen nachvollziehbar zu machen. Dies wird oft über Gespeicherte Prozeduren (Stored Procedures) in der SQL-Datenbank umgesetzt. Zudem werden hier Datenbeziehungen über Primär- und Fremdschlüssel definiert.
4. **Bizcore-Schicht (Business Core):** Diese Schicht enthält die eigentliche Geschäftslogik. Hier werden Daten aus verschiedenen Vorsystemen zu einer einheitlichen Sicht zusammengeführt (z.B. ein "Konzern-Kundenstamm") und anwendungsspezifische Berechnungen durchgeführt. Beispielsweise kann hier der Deckungsbeitrag für Verkäufe ermittelt werden, indem auf historisierte Kostenkomponenten (wie den Konzern-Einstandspreis zum Belegdatum) aus der Core-Schicht zurückgegriffen wird. Die Transformationen in dieser Schicht erfolgen ebenfalls häufig über Gespeicherte Prozeduren.
5. **Datamart-Schicht:** Die oberste Schicht stellt spezifische Datensichten für definierte Anwendungsbereiche oder Nutzergruppen bereit (z.B. ein Pricing-Datamart). Die Daten sind hier oft in einem dimensional Modell (z.B. Sternschema) aufbereitet, das für Analysewerkzeuge wie Power BI oder SSAS Tabular Models optimiert ist. Diese Schicht bildet die direkte Grundlage für Reports und Dashboards.

Der gesamte ELT-Prozess wird technologisch durch Microsoft SQL Server Integration Services (SSIS) für die Transformationslogik (teilweise direkt in SSIS, teilweise durch Aufruf von Stored Procedures in der Datenbank) und durch die Azure Data Factory für die Orchestrierung (zeitliche Steuerung der Lade- und Transformationsprozesse, z.B. nächtliche Läufe) unterstützt.

Die Kenntnis dieses Schichtenmodells und der zugrundeliegenden Prozesse ist essenziell für den Aufbau einer Data Science Umgebung.

3.2.2 Maßnahmen zur Sicherstellung der Datenqualität (z.B. Abgleich Pacvue vs. interne Daten)

Im Rahmen dieser Studienarbeit und der prototypischen Umsetzung konnte aufgrund fehlenden Zugriffs auf die produktive Quellsysteme des international tätigen, mittelständischen Branchenführers sowie zur Vereinfachung nicht die volle Tiefe der beschriebenen Ziel-Architektur abgebildet werden. Stattdessen wurde das DWH-Konzept wie folgt simuliert:

- **Datenbank:** Eine einzelne Datenbank (PricingPrototypeDB) wurde auf einer lokalen SQL Server Developer Edition Instanz erstellt. Diese diente als Container für alle Tabellen des Prototyps.
- **Quellen & Staging/Cleanse:** Synthetische Beispieldaten wurden in Form von CSV-Dateien erstellt. Das Laden dieser Daten in die SQL-Tabellen (z.B. dbo.Produkte, dbo.Verkaeufe, dbo.Wettbewerbspreise) mittels einfacher SQL-Skripte (BULK INSERT) oder des SSMS Import/Export-Assistenten simulierte den Staging-Prozess und grundlegende Cleansing-Aspekte (durch die angenommene Qualität der synthetischen Daten).
- **Core/Bizcore:** Die Integration der Daten erfolgte durch das Definieren von Tabellenstrukturen und Beziehungen direkt in der SQL-Datenbank bzw. später im Datenmodell von Power BI. Komplexe Transformationen und Berechnungen, die typischerweise in der Bizcore-Schicht stattfinden (z.B. Feature Engineering), wurden im Prototyp primär in Python durchgeführt (siehe Kapitel 3.3). Eine explizite Historisierung (SCD II) wurde im SQL-Teil des Prototyps nicht implementiert.
- **Datamart:** Die für die Analyse in Power BI genutzten Tabellen, inklusive der Tabelle mit den Modellergebnissen (dbo.ModelOutput), können als Simulation der Datamart-Schicht betrachtet werden, da sie die aufbereitete Datengrundlage für den spezifischen Anwendungsfall (Pricing-Analyse und -Modellierung) darstellten.
- **ELT-Werkzeuge:** Statt SSIS und Azure Data Factory wurden für den Prototyp Python (pandas) für Datenaufbereitung und -ladung sowie der SSMS Import/Export-Assistent verwendet. Die Ausführung erfolgte manuell.

Diese Simulation erlaubte es, den konzeptionellen Datenfluss von der Quelle über eine (vereinfachte) zentrale Speicherung und Transformation bis hin zur Analyse nachzubilden, auch wenn die technische Ausprägung der einzelnen DWH-Schichten stark vereinfacht wurde.

3.3 Feature Engineering mittels Python (pandas, numpy) und SQL

3.3.1 Erstellung relevanter Merkmale (z.B. Zeitmerkmale, Preisindizes, Marketing-Einflüsse, rollierende Kennzahlen)

Das Feature Engineering stellt einen zentralen Schritt in der Datenvorbereitung für die Preismodellierung dar. Aus den Rohdaten werden dabei aussagekräftige Merkmale abgeleitet, die dem Modell relevante Informationen über Preis-Mengen-Zusammenhänge und deren Einflussgrößen bereitstellen. Zu den wichtigsten Merkmalsgruppen zählen Zeitmerkmale (Wochentag, Monat, Quartal, Jahreszeit, Feiertags-Flags), die saisonale Nachfragemuster abbilden; Preisindizes, die den eigenen Preis ins Verhältnis zum Wettbewerbspreis setzen (relativer Preisindex); Marketing-Einflüsse, die Werbeaktivitäten und Klickpreise in den Erklärungsrahmen einbeziehen; sowie rollierende Kennzahlen wie gleitende Durchschnitte der Verkaufsmenge oder des Preises über definierte Zeitfenster, die kurzfristige Trendkomponenten erfassen.

Im Prototyp wurde das Feature Engineering mit Hilfe von Python (Bibliotheken pandas und numpy) auf den geladenen DataFrames umgesetzt. Die Funktion `build_feature_dataframe()` in `src/data_preparation.py` konsolidiert die Daten aus den verschiedenen Quellen und berechnet die für die Modellierung benötigten Merkmale, darunter insbesondere den logarithmierten Preis sowie Zeitkomponenten, die eine robustere Schätzung der Preiselastizität ermöglichen.

3.3.2 Transformation von Daten für Modellierungszwecke (z.B. Log-Transformation)

Für eine stabile und interpretierbare Regressionsmodellierung sind bestimmte Datentransformationen erforderlich. Von besonderer Bedeutung ist die Log-Transformation von Preis und Verkaufsmenge: Durch die doppelt-logarithmische Modellformulierung (log-log-Regression) wird der Regressionskoeffizient des Preises direkt als Preiselastizität interpretierbar – er gibt an, um wieviel Prozent sich die Nachfrage bei einer einprozentigen Preisänderung verändert. Diese Transformation linearisiert zudem die typischerweise multiplikative Preis-Mengen-Beziehung und verbessert die Modellanpassung. Ergänzend können Skalierungsmaßnahmen (z.B. Standardisierung numerischer Features) sowie die Kodierung kategorialer Merkmale (z.B. One-Hot-Encoding für Produktkategorien) erforderlich sein.

Im Prototyp wurden diese Transformationen direkt in Python vorgenommen. Die Konvertierung von Datumsspalten, die Berechnung logarithmischer Werte und einfache Ableitungen wie der relative Preisindex wurden dabei als vorbereitende Berechnungsschritte im Feature-Engineering-Modul implementiert, bevor die Daten dem Modellierungsprozess übergeben wurden.

4. Entwicklung der Machine-Learning-Modelle zur Preisoptimierung (ca. 3-4 Seiten) [Modeling]

4.1 Auswahl geeigneter Modellierungsansätze

Für die Modellierung im Rahmen der dynamischen Preisoptimierung kommen grundsätzlich zwei Klassen von Ansätzen in Betracht. Modelle zur Schätzung der Preiselastizität – insbesondere Regressionsmodelle in log-log-Spezifikation – quantifizieren den direkten Zusammenhang zwischen Preisänderungen und der resultierenden Nachfragereaktion und liefern damit die theoretische Basis für optimale Preisempfehlungen gemäß der Amoroso-Robinson-Relation. Modelle zur Nachfrageprognose, wie etwa Zeitreihenmodelle (ARIMA, Prophet) oder gradientenbasierte Ensemble-Verfahren (z.B. Random Forest, XGBoost), ergänzen diesen Ansatz, indem sie die absolute Nachfrage unter gegebenen Marktbedingungen vorhersagen – insbesondere nützlich, wenn keine ausreichenden Preisvarianzen für eine Elastizitätsschätzung vorliegen.

Für den Prototyp dieser Studienarbeit wurde bewusst der Fokus auf die Schätzung der Preiselastizität mittels einer (vereinfachten) linearen Regression in log-log-Spezifikation gelegt. Dieser Ansatz ist methodisch transparent, direkt interpretierbar und erlaubt es, den gesamten Modellierungsprozess – von der Datenaufbereitung über das Training bis hin zur Ableitung von Preisempfehlungen – vollständig mit Python und scikit-learn abzubilden, wie in den folgenden Kapiteln detailliert beschrieben wird.

4.2 Implementierung der Modelle in Python:

4.2.1 Nutzung von Bibliotheken

Die Implementierung der Modelle im Prototyp basiert auf dem etablierten Python-Ökosystem für Data Science und Machine Learning. Die Bibliothek scikit-learn wurde für die eigentliche Modellierung eingesetzt: Sie stellt sowohl die Regressionsklassen (z.B. LinearRegression) als auch standardisierte Methoden für die Datenaufteilung (`train_test_split`) und die Berechnung von Evaluationsmetriken (`r2_score`, `mean_absolute_error`, `mean_squared_error`) bereit. Die Datenhaltung, -manipulation und das Feature Engineering erfolgten mit pandas (DataFrames, Merging, Aggregation), während numpy für numerische Operationen wie die Berechnung von Logarithmen und Matrizenoperationen genutzt wurde.

Als Entwicklungsumgebung diente Jupyter Notebook (bereitgestellt über die Anaconda-Distribution), das eine iterative und interaktive Arbeitsweise ermöglicht – insbesondere für explorative Analysen und die schrittweise Entwicklung und Visualisierung des Modellierungsprozesses. Das vollständige End-to-End-Notebook ist unter `notebooks/dynamic_pricing_prototype.ipynb` einsehbar. Die Modellierungslogik ist zudem in das Skript `src/model.py` überführt worden, das eine reproduzierbare und modular einsetzbare Implementierung darstellt.

4.2.2 Umgang mit produkt- oder kategoriespezifischen Unterschieden

Die Berücksichtigung produkt- oder kategoriespezifischer Unterschiede in der Preiselastizität stellt eine zentrale Modellierungsentscheidung dar. Grundsätzlich stehen zwei Strategien zur Verfügung: Separate Modelle je Produkt oder Kategorie ermöglichen eine maximale Anpassungsfähigkeit an die jeweiligen Preis-Mengen-Charakteristika, erfordern jedoch ausreichende Datenpunkte je Segment und erhöhen den Verwaltungsaufwand erheblich. Alternativ kann ein einheitliches Modell mit der Produkt-ID oder Kategorie als kategorialem Feature (z.B. nach One-Hot-Encoding) trainiert werden, das Elastizitätsunterschiede implizit durch die geschätzten Koeffizienten abbildet und mit weniger Daten auskommt.

Im Prototyp wurde ein pragmatischer Ansatz gewählt: Da die synthetischen Daten für jeden der fünf Produkte eine ausreichende Anzahl an Beobachtungen enthielten, wurden separate Modelle je Produkt trainiert. Dies erlaubt eine direkte, produktspezifische Interpretation der geschätzten Elastizitätskoeffizienten und vermeidet mögliche Verzerrungen durch eine zu starke Pooling-Annahme über heterogene Produktkategorien hinweg.

4.3 Training, Validierung und Auswahl der Modelle

4.3.1 Aufteilung in Trainings-, Validierungs- und Testdaten

Eine methodisch korrekte Aufteilung der Daten in Trainings- und Testmengen ist eine Grundvoraussetzung für eine verlässliche Modellbewertung. Bei Zeitreihendaten – und Verkaufsdaten sind inhärent zeitlich geordnet – ist die klassische zufällige Hold-out-Aufteilung ungeeignet, da sie temporale Abhängigkeiten verletzt und zu einer unrealistisch optimistischen Modellbewertung führen kann (Data Leakage). Stattdessen wird ein Zeitreihen-Split verwendet: Die chronologisch früheren Beobachtungen bilden die Trainingsmenge, während die neuesten Beobachtungen zur Evaluation herangezogen werden. Dieses Vorgehen simuliert reale Bedingungen, bei denen das Modell auf der Basis vergangener Daten trainiert wird und anschließend Vorhersagen für zukünftige Perioden treffen soll.

Im Prototyp wurde eine Aufteilung von 80 % Trainingsdaten und 20 % Testdaten nach dem Zeitreihen-Split-Prinzip vorgenommen. Die Implementierung erfolgte in Python (scikit-learn), wobei sichergestellt wurde, dass die temporale Reihenfolge der Datenpunkte für jeden Produktdatensatz erhalten blieb.

4.3.2 Metriken zur Bewertung der Modellgüte

Zur Bewertung der Modellgüte wurden für die im Prototyp eingesetzten Regressionsmodelle die in der Literatur etablierten Metriken verwendet. Das Bestimmtheitsmaß R^2 (Coefficient of Determination) gibt an, welcher Anteil der Varianz der Zielvariablen (log. Absatzmenge) durch das Modell erklärt wird; ein Wert nahe 1 indiziert eine hohe Anpassungsgüte. Der Mean Absolute Error (MAE) misst den durchschnittlichen absoluten Prognosefehler in der Einheit der Zielvariablen und ist robust gegenüber Ausreißern. Der Root Mean Squared Error (RMSE) bestraft große Einzelfehler stärker und erlaubt daher eine differenziertere Beurteilung der Modellstabilität.

Alle drei Metriken wurden mit den Funktionen `r2_score`, `mean_absolute_error` und `mean_squared_error` aus dem Modul `sklearn.metrics` auf dem jeweiligen Testdatensatz berechnet. Die konkreten Metrikwerte für alle fünf Produkte des Prototyps sind in `MODELL_DOKUMENTATION.md` → Kap. 7.1 dokumentiert.

4.4 Ableitung von Preisempfehlungen aus den Modellergebnissen

Die Überführung von Modelloutputs in operative Preisempfehlungen ist der eigentliche präskriptive Schritt im Optimierungsprozess. Konzeptionell bildet hierfür die Amoroso-Robinson-Relation die theoretische Grundlage: Der gewinnmaximierende Preis lässt sich als Funktion der geschätzten Preiselastizität und der Grenzkosten ableiten. In der praktischen Umsetzung müssen jedoch zusätzliche Randbedingungen berücksichtigt werden, darunter Mindest- und Höchstpreise (z.B. aus Margenzielen oder vertraglichen Vereinbarungen), Wettbewerbspreise als Orientierungsgröße sowie plattformspezifische Restriktionen auf Amazon. Diese Business Rules stellen sicher, dass die algorithmisch ermittelten Preise betriebswirtschaftlich sinnvoll und marktkonform sind.

Im Prototyp wurde ein vereinfachter Ansatz zur Preisempfehlung implementiert: Der geschätzte Regressionskoeffizient für den logarithmierten Preis wird als Preiselastizität ϵ interpretiert. Basierend auf diesem Wert und dem aktuellen Wettbewerbspreis berechnet die Funktion `recommend_price()` in `src/pricing_optimizer.py` einen Preisvorschlag: Bei hochelastischer Nachfrage ($|\epsilon| > 1$) wird eine Annäherung an den Wettbewerbspreis empfohlen, während bei unelastischer Nachfrage ein Aufschlag möglich ist. Die resultierenden Ergebnisse – Elastizität, Ist-Preis, Wettbewerbspreis und Preisempfehlung je Produkt – werden abschließend in der SQL-Datenbanktabelle `dbo.ModelOutput` gespeichert und stehen damit für das Reporting in Power BI zur Verfügung.

5. Evaluation und Validierung der Preisstrategie (ca. 2 Seiten) [Evaluation]

5.1 Technische Evaluation der Modellergebnisse und Elastizitäten

Die technische Evaluation des Modells beginnt mit der Analyse der in Kapitel 4.3.2 definierten Gütemetriken auf dem Testdatensatz. Die R^2 -Werte der produktspezifischen Modelle geben Aufschluss darüber, inwieweit der logarithmierte Preis als wesentlicher Erklärungsfaktor die Varianz der Absatzmenge beschreibt; MAE und RMSE quantifizieren die typische Prognosegüte in der log-transformierten Einheit. Die im Prototyp erzielten Werte sind in `MODELL_DOKUMENTATION.md` → Kap. 7 dokumentiert.

Ergänzend zur quantitativen Modellbewertung ist eine inhaltliche Plausibilitätsprüfung der geschätzten Elastizitäten unerlässlich. Ökonomisch fundierte Erwartungen sind: Preiselastizitäten sollten negativ sein (steigende Preise führen zu sinkender Nachfrage), ihre Größenordnung sollte im Bereich üblicher Konsumgüter liegen (typischerweise $-0,5$ bis $-2,0$), und Produkte mit stärkerem Wettbewerbsdruck sollten höhere Elastizitäten aufweisen als Nischenprodukte. Abweichungen von diesen Erwartungen würden auf Modellierungsprobleme, Datenfehler oder unzureichende Erklärungsvariablen hinweisen und erfordern eine kritische Auseinandersetzung mit den zugrundeliegenden Ursachen.

5.2 Konzeption und Simulation von A/B-Tests im E-Commerce Kontext

A/B-Tests – auch als kontrollierte Feldexperimente bezeichnet – stellen das methodisch robusteste Instrument zur Validierung von Preisänderungen im realen Marktumfeld dar. Während Modellmetriken nur aussagen, wie gut das Modell historische Muster nachbildet, liefern A/B-Tests kausale Evidenz dafür, ob eine Preisanpassung tatsächlich die erwartete Wirkung auf Absatz, Umsatz und Marge erzielt. Für einen international tätigen, mittelständischen Branchenführer auf Amazon könnte ein solches Testdesign wie folgt aussehen: Eine zufällig ausgewählte Teilmenge von Produkten (Testgruppe) erhält den modellempfohlenen Preis, während eine vergleichbare Kontrollgruppe den bestehenden Preis beibehält. Nach einem definierten Testzeitraum werden die Kennzahlen beider Gruppen statistisch verglichen, um den kausalen Effekt der Preisänderung zu isolieren.

Die Ergebnisse solcher Tests können auf mehreren Ebenen zur Modellverbesserung genutzt werden: Erstens liefern sie echte Reaktionsdaten, die das Modell mit tatsächlich beobachteten Elastizitäten kalibrieren; zweitens decken sie Produktsegmente auf, in denen das Modell systematisch über- oder unterschätzt; drittens ermöglichen sie eine laufende Überprüfung der Modellhypothesen unter sich verändernden Marktbedingungen. Im Rahmen des vorliegenden Prototyps konnte ein A/B-Test aufgrund fehlender realer Verkaufsdaten und mangelnder Plattformzugriffe nur konzeptionell behandelt werden; die methodische Grundlage ist in `MODELL_DOKUMENTATION.md` → Kap. 7.3 dokumentiert.

5.3 Bewertung der Auswirkungen auf die Geschäftsziele (Umsatz, Marge)

Die Messung des geschäftlichen Impacts eines operativen Preisoptimierungssystems erfordert geeignete Methoden, die über die rein technische Modellbewertung hinausgehen. Im Live-Betrieb bieten sich insbesondere der Vergleich von Test- und Kontrollgruppen (bei parallelen A/B-Tests) sowie Zeitreihenvergleiche (Before/After-Analysen mit statistischer Kontrolle von Confoundern) an. Relevante KPIs sind dabei Umsatzveränderung (ΔUmsatz), Margenentwicklung (ΔMarge) und Absatzmengendelta (ΔMenge) im Vergleich zur Baseline-Periode oder Kontrollgruppe.

Im Prototyp wurde dieser Impact-Nachweis anhand synthetischer Daten in Power BI (mittels DAX-Measures) simuliert. Für jedes Produkt wurden auf Basis der Modellergebnisse aus `dbo.ModelOutput` die erwarteten Veränderungen in Umsatz und Marge berechnet, wenn der empfohlene Preis anstelle des Ist-Preises angewendet wird. Die konkreten Berechnungsformeln und die daraus resultierenden ΔUmsatz - und ΔMarge -Werte für alle Produkte sind in `MODELL_DOKUMENTATION.md` → Kap. 6.3 detailliert dokumentiert.

5.4 Iterative Verbesserung der Modelle basierend auf Evaluationsergebnissen

Ein datengetriebenes Preisoptimierungsmodell ist kein statisches Artefakt, sondern erfordert einen kontinuierlichen Verbesserungsprozess, um dauerhaft valide Empfehlungen zu liefern. Marktbedingungen, Wettbewerbsverhalten und Nachfragestrukturen verändern sich im Zeitverlauf; ein Modell, das ausschließlich auf historischen Daten trainiert wurde, verliert daher schrittweise an Prognosegenauigkeit (Concept Drift). Um dem entgegenzuwirken, sind ein regelmäßiges Monitoring der Modellgüte (z.B. wöchentliche Überprüfung von MAE und R^2 auf neuen Daten), definierte Trigger für ein Retraining (z.B. bei Unterschreitung eines Mindest- R^2) sowie eine periodische Feature-Anpassung an veränderte Einflussfaktoren notwendig. Dieses Prinzip entspricht dem CRISP-DM-Prozessmodell, das in Kapitel 1 beschrieben ist und den iterativen Charakter von Data-Mining-Projekten betont.

6. Operationalisierung und Monitoring im BI-System (ca. 3 Seiten) [Deployment]

6.1 Bereitstellung der Modellergebnisse

Die Bereitstellung der Modellergebnisse für das operative Reporting und die Entscheidungsunterstützung ist ein zentraler Schritt im Deployment-Prozess. Im Zielsystem ist vorgesehen, die berechneten Elastizitäten und Preisempfehlungen in das Data Warehouse zu schreiben, von wo aus sie über die Datamart-Schicht für Analysewerkzeuge wie Power BI oder SSAS Tabular Models zugänglich gemacht werden. Diese Architektur ermöglicht eine saubere Trennung zwischen dem Modellierungsprozess (Python/scikit-learn) und dem Reporting-Frontend (Power BI), da beide Schichten über die SQL-Datenbank entkoppelt sind.

Im Prototyp wurde dieser Ansatz durch eine direkte Speicherung der Modellergebnisse in der Tabelle `dbo.ModelOutput` der lokalen SQL Server Developer Edition Datenbank umgesetzt. Die Funktion `save_recommendations()` in `src/pricing_optimizer.py` schreibt für jedes Produkt die berechneten Kennzahlen (Elastizität, Ist-Preis, Wettbewerbspreis, empfohlener Preis, erwartete Mengen- und Umsatzveränderung) in diese Tabelle, die anschließend als Datenquelle für das Power BI Dashboard dient.

6.2 Aufbau eines analytischen Datenmodells (Tabular Model in SSAS) für das Frontend

6.2.1 Definition von Dimensionen, Fakten und Hierarchien für Pricing-Analysen

Im Zielsystem ist der Einsatz von SQL Server Analysis Services (SSAS) Tabular als analytische Datenbankschicht vorgesehen. SSAS Tabular-Modelle bieten durch die zugrundeliegende VertiPaq-Komprimierungstechnologie hochperformante Analysen über große Datenmengen, ein zentrales, versioniertes Datenmodell mit einheitlichen Business-Metriken sowie eine nahtlose Integration mit Power BI und Excel. Die Kenntnis von SSAS Tabular und DAX stellt dabei eine wertvolle Kompetenz im Bereich Enterprise Business Intelligence dar, da Tabular-Modelle in großen Organisationen als Single Source of Truth für sämtliche Reporting-Bedarfe fungieren.

Da Power BI Desktop intern die gleiche VertiPaq-Engine wie SSAS Tabular verwendet, diente das im Prototyp aufgebaute Power BI-Datenmodell als funktional äquivalente Simulation eines Tabular

Models. Das Modell folgt dabei einer Sternschema-Struktur: Die Faktentabellen (Verkäufe, Wettbewerbspreise, Modellergebnisse) sind über definierte Beziehungen mit den Dimensionstabellen (Produkte, Datum) verknüpft. Diese Modellstruktur ermöglicht effiziente Filter- und Aggregationsoperationen über mehrere Dimensionen hinweg und bildet die Grundlage für alle DAX-Berechnungen im Prototyp.

6.2.2 Implementierung relevanter Kennzahlen mittels DAX

Die Aussagekraft eines Power BI Dashboards hängt wesentlich von der Qualität und Korrektheit der definierten DAX-Measures ab. Im Rahmen des Prototyps wurden zentrale Kennzahlen für das Pricing-Reporting implementiert. Das Measure Turnover (Umsatz) berechnet den Gesamtumsatz als Summe aus Verkaufsmenge multipliziert mit dem Verkaufspreis. Die Net Margin (simuliert) ergibt sich aus der Differenz von Umsatz und simulierten Einstandsdaten, bezogen auf den Umsatz, und gibt die prozentuale Rotertragsmarge wieder. Durchschnittspreis ermittelt den mengengewichteten Mittelwert der erzielten Verkaufspreise über einen gewählten Zeitraum. Weitere Measures zeigen die vom Modell berechnete Preiselastizität und den empfohlenen Preis aus `dbo.ModelOutput` in Abhängigkeit der Produktauswahl an und ermöglichen so einen direkten Vergleich zwischen Ist-Zustand und Modellempfehlung im Dashboard. Die korrekte Implementierung dieser Measures erfordert ein solides Verständnis von DAX-Evaluierungskontext und Filterlogik und verdeutlicht die Bedeutung von BI-Entwicklungskompetenzen im Rahmen eines solchen Projekts.

6.3 Entwicklung von Power BI Dashboards und Reports

6.3.1 Visualisierung von Preisempfehlungen und deren Einflussfaktoren

Der Power BI Prototyp umfasst mehrere Berichtsseiten, die unterschiedliche Aspekte der dynamischen Preisoptimierung visualisieren. Eine Übersichtsseite zeigt die Entwicklung von Umsatz und Absatzmenge im Zeitverlauf, untergliedert nach Produkten und Kategorien, und ermöglicht einen schnellen Überblick über die Geschäftsentwicklung. Eine dedizierte Pricing-Analyseseite stellt die berechneten Preiselastizitäten je Produkt dar – etwa als Balkendiagramm –, zeigt den Ist-Preis im Vergleich zum Wettbewerbspreis sowie den vom Modell empfohlenen Preis und die erwarteten Auswirkungen auf Menge, Umsatz und Marge. Eine Wettbewerbspreisseite visualisiert die zeitliche Entwicklung der eigenen Preise im Vergleich zu den simulierten Marktpreisen und identifiziert Perioden, in denen Preisanpassungen besonders relevant wären. Diese Visualisierungen machen die abstrakten Modelloutputs für das Management und das E-Commerce-Team unmittelbar verständlich und handlungsrelevant.

6.3.2 Monitoring der Preisentwicklung und Modell-Performance

Das Prototyp-Dashboard erfüllt auch die Funktion eines Monitoring-Instruments für die operative Steuerung von KPIs und Modelloutputs. Durch die Einbindung der Tabelle `dbo.ModelOutput` in das Power BI-Datenmodell sind Elastizitäten, Preisempfehlungen und simulierte Ergebniskennzahlen stets aktuell abrufbar und können produktübergreifend verglichen werden. Verantwortliche des Amazon-Teams können so auf einen Blick erkennen, für welche Produkte das Modell eine Preisanpassung empfiehlt, welche Auswirkungen zu erwarten sind und ob die tatsächliche Preisentwicklung im Einklang mit den Modellvorgaben steht.

6.3.3 Bereitstellung von Analysewerkzeugen für das Amazon-Team und Controlling

Der Power BI Prototyp positioniert sich dabei als konkretes Beispiel für ein self-service-fähiges Analysewerkzeug, das sowohl dem Amazon-Team für operative Preisentscheidungen als auch dem

Controlling für die strategische Ergebnissteuerung dient. Power BI bietet durch seine intuitiv bedienbare Oberfläche und die Möglichkeit zur Integration verschiedenster Datenquellen eine niedrigschwellige Zugangsmöglichkeit zu komplexen Analyseergebnissen, ohne dass Endanwender über tiefgehende technische Kenntnisse verfügen müssen. In einem produktiven Einsatz würde das Dashboard über den Power BI Service bereitgestellt, sodass alle berechtigten Mitarbeiter stets auf aktuelle Daten zugreifen können.

6.4 Überlegungen zur Automatisierung des Gesamtprozesses

Die Automatisierung des Gesamtprozesses ist eine notwendige Voraussetzung für den operativen Einsatz des Preisoptimierungssystems. Im Zielsystem sind hierfür verschiedene Maßnahmen vorgesehen: Die Datenaktualisierung (nightly ETL-Läufe) kann durch den SQL Server Agent oder die Azure Data Factory orchestriert werden. Das regelmäßige Modell-Retraining lässt sich als geplanter Python-Skript-Job ausführen, der auf neuen Daten das Modell neu kalibriert und die Ergebnisse in `dbo.ModelOutput` fortschreibt. Die Aktualisierung des Power BI Dashboards erfolgt im Zielsystem über den Power BI Scheduled Refresh, der sicherstellt, dass Endanwender stets auf aktuelle Modelloutputs zugreifen können.

Im Prototyp dieser Studienarbeit wurde der gesamte Prozess – von der Datengenerierung über das Python-Modellierungsskript bis hin zur manuellen Aktualisierung des Power BI-Berichts – ausschließlich manuell ausgeführt. Eine Automatisierung war für den Nachweis der methodischen Machbarkeit nicht erforderlich, ist aber als konsequente nächste Ausbaustufe im Zielsystem konzeptionell vorgesehen.

7. 7. Zusammenfassung und Ergebnisse

7.1 Darstellung des entwickelten Systems

Der im Rahmen dieser Studienarbeit entwickelte Prototyp realisiert einen vollständigen End-to-End-Datenfluss für ein dynamisches Preisoptimierungsmodell. Ausgehend von synthetischen Produktstamm-, Verkaufs- und Wettbewerbspreisdaten, die mittels Python generiert und im CSV-Format bereitgestellt wurden, erfolgt zunächst das Laden in eine lokale SQL Server Developer Edition Datenbank. Aus dieser relationalen Datenbank werden die Daten von einem Python-basierten Modellierungsmodul (pandas, numpy, scikit-learn) abgerufen, feature-engineered und für das Training produktspezifischer Elastizitätsregressionsmodelle verwendet. Die berechneten Preisempfehlungen werden anschließend zurück in den SQL Server (Tabelle `dbo.ModelOutput`) geschrieben. Abschließend verbindet sich Power BI Desktop mit der SQL Server Datenbank, liest den `ModelOutput` sowie die Basistabellen und stellt die Ergebnisse in interaktiven Dashboards dar. Dieser Prozess bildet die wesentlichen Architekturkomponenten eines produktiven Systems ab und demonstriert die technische Machbarkeit des Gesamtkonzepts.

7.2 Zusammenfassung der Kernergebnisse

Die wichtigsten Ergebnisse des Prototyps lassen sich anhand der berechneten Preiselastizitäten und der daraus abgeleiteten Preisempfehlungen zusammenfassen. Für die fünf simulierten Produkte wurden Elastizitäten zwischen $-0,52$ (Produkt P004, unelastische Nachfrage) und $-1,32$ (Produkt P003, hochelastische Nachfrage) ermittelt, was sowohl ökonomisch plausible Vorzeichen als auch realistische Größenordnungen für Konsumgüter aufweist. Die R^2 -Werte der produktspezifischen Regressionsmodelle liegen zwischen $0,42$ und $0,61$, was angesichts des simulierten Datencharakters eine akzeptable Modellgüte darstellt und das Potenzial eines solchen Ansatzes auf echten Produktionsdaten verdeutlicht. Die vollständige Ergebnistabelle mit allen Produkten und den

simulierten KPI-Auswirkungen (Δ Menge, Δ Umsatz, Δ Marge) ist in MODELL_DOKUMENTATION.md → Kap. 6.4 dokumentiert und im nachfolgenden Abschnitt dieser Studienarbeit zusammenfassend dargestellt.

Die wichtigsten Ergebnisse des Prototyps (Produktionslauf mit SEED=42, Verkaufsjahr 2023):

Produkt	ϵ	R^2	Ist-Preis	Wettbewerbspr	Empf. Preis	Δ Menge	Δ Umsatz	Δ Marge
P001	-1,18	0,61	63,57 €	62,67 €	76,28 €	-23,6 %	-8,4 %	+5,3 %
P002	-1,05	0,42	102,71 €	98,27 €	104,97 €	-2,3 %	-0,2 %	+1,9 %
P003	-1,32	0,57	24,63 €	22,99 €	29,56 €	-26,4 %	-11,6 %	-1,6 %
P004	-0,52	0,50	30,39 €	38,72 €	33,95 €	-6,2 %	+4,9 %	+15,6 %
P005	-0,66	0,44	45,37 €	49,96 €	48,33 €	-4,3 %	+2,0 %	+6,9 %

7.3 Beitrag zur datengesteuerten Entscheidungsfindung und Erreichung strategischer Ziele

Das entwickelte System demonstriert eindrücklich, wie datengesteuerte Entscheidungsfindung im E-Commerce-Pricing operationalisiert werden kann. Anstelle von intuitiven Einzelentscheidungen oder starren Preisregeln ermöglicht der modellbasierte Ansatz eine systematische, skalierbare und nachvollziehbare Preisgestaltung, die Wettbewerbsdynamiken, Nachfrageelastizitäten und Margenziele gleichzeitig berücksichtigt. Für einen international tätigen, mittelständischen Branchenführer bedeutet dies konkret: Das Amazon-Team erhält für jedes Produkt eine datenbasierte Preisempfehlung mit quantifizierten Erwartungseffekten auf Umsatz und Marge, die als fundierte Entscheidungsgrundlage dienen und die bisherigen manuellen Prozesse ersetzt oder zumindest substantiell unterstützt. Der Prototyp zeigt, dass die dafür notwendigen Technologien und Methoden bereits mit einem überschaubaren Mitteleinsatz prototypisch implementiert werden können.

7.4 Ausblick auf mögliche Weiterentwicklungen und Limitationen

Die vorliegende Arbeit unterliegt als Prototyp naturgemäß einer Reihe von Limitationen, die offen benannt werden müssen. Erstens basiert die gesamte Modellierung auf synthetischen Daten, die zwar realistische Preis-Mengen-Zusammenhänge simulieren, jedoch nicht die Komplexität und Heterogenität echter Produktionsdaten widerspiegeln können. Zweitens wurden bewusst einfache Regressionsmodelle eingesetzt; in der Praxis könnten nichtlineare Ansätze (Random Forest, XGBoost) oder Zeitreihenmodelle (Prophet, ARIMA) eine deutlich bessere Prognosequalität liefern. Drittens wurden keine echten A/B-Tests durchgeführt, sodass die kausalitätsstiftende Validierung der Preisempfehlungen aussteht. Viertens ist der Prototyp manuell ausgeführt und nicht in eine produktive Automatisierungsinfrastruktur eingebettet.

Die Weiterentwicklungspotenziale des Systems sind entsprechend vielfältig. Prioritär ist der Übergang auf echte Verkaufs- und Wettbewerbsdaten aus den produktiven Quellsystemen. Darüber hinaus bieten sich die Implementierung von A/B-Test-Frameworks auf Amazon, der Einsatz komplexerer

Modellierungsansätze, die vollständige Automatisierung der Pipelines (Azure Data Factory, SQL Server Agent, Power BI Scheduled Refresh) sowie die Nutzung von Cloud-Diensten (z.B. Azure Machine Learning) als konkrete nächste Schritte an. Die Umsetzung dieser Schritte erfordert zwingend eine dedizierte Data Scientist Umgebung mit entsprechendem Datenzugang, technischer Infrastruktur und interdisziplinärer Zusammenarbeit zwischen E-Commerce, IT und Controlling.

Abbildungsverzeichnis

Figure 1-1 Cross Industry Standard Process for Data Mining (CRISP-DM), Quelle: Wikipedia, CC BY-SA 3.0.....	8
---	---

Literaturverzeichnis

IBCS Association, International Business Communication Standards (IBCS), Version 1.2

Microsoft Learn Portal, Datentypen (Transact-SQL), <https://learn.microsoft.com/de-de/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver16>, (abgerufen am 04.03.2024)

Microsoft Learn Portal, „PowerBI usage scenarios“, <https://learn.microsoft.com/en-us/power-bi/guidance/powerbi-implementation-planning-usage-scenario-overview>, (abgerufen am 04.03.2024)

Microsoft Learn Portal, „PowerBI usage scenarios: Customizable managed self-service BI“, <https://learn.microsoft.com/en-us/power-bi/guidance/powerbi-implementation-planning-usage-scenario-customizable-managed-self-service-bi>, (abgerufen am 04.03.2024)